

Ejercicio [6 puntos]

El CNI quiere desarrollar una aplicación para espiar los movimientos de la población, así como los diferentes encuentros que tienen las personas de su interés entre sí. Esta aplicación registra los acercamientos entre personas, la fecha en la que se ha producido y el tiempo que dura dicho encuentro.

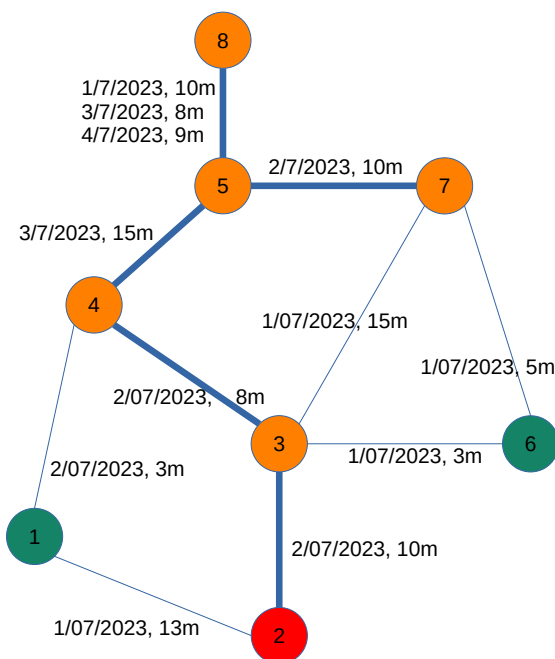
Se desea construir una aplicación para marcar a personas sospechosas. Para ello se parte de una persona de interés y se irán marcando como personas sospechosas aquellas que hayan mantenido un contacto con la persona de interés durante más de M minutos. Esto se irá propagando, de tal manera que una persona que mantenga contacto con una persona marcada como sospechosa por más de M minutos también será marcada como sospechosa, y así sucesivamente. Todo ello a partir de una fecha F dada. Así, el árbol de interés de una persona R estará formado por esa persona y toda persona P tal que exista una persona X que ya está en el árbol y con la que P se ha relacionado durante más de M minutos tras la fecha F.

En la figura adjunta se presenta el árbol (naranja) que parte del individuo 2 (rojo) para fecha igual o posterior al 2/7/2023 y por más de 5 minutos. Obsérvese que el nodo 5 y el 8 se han visto 3 veces en diferentes días.

Para obtener dicho árbol, se desea implementar una clase `InteractionDetector` que permita almacenar todas las interacciones entre personas y que permita obtener el árbol de interés de la persona R (que será la raíz del árbol) durante más de M minutos a partir de una fecha F. Para construir dicha aplicación se propone crear la clase `InteractionDetector`.

Dicha clase tendrá los siguientes métodos:

- `storeContact(Date date, int minutes, int idPersonA, int idPersonB)`. Este método permitirá a la clase almacenar todas las interacciones entre personas. Dicha información la guardará para luego poder realizar consultas.
- `Collection<Integer> getInterestTree(int idRoot, Date date, int minutes)`. Este método permitiría obtener el árbol de interés de `idRoot` (que será la raíz del árbol) a partir de una fecha indicada y durante más de los minutos indicados.



Se pide:

- a) **[0,5 puntos]** Añadir las propiedades que se consideren oportunas a las clases `InteractionDetector` para cumplir el objetivo deseado.
- b) **[1 punto]** Implementar en `InteractionDetector` el método `storeContact` de acuerdo con las propiedades añadidas.

c) **[2 puntos]** Implementar el método `getInterestTree` de acuerdo con las propiedades añadidas

Para facilitar la creación de la clase `InteractionDetector` es imprescindible implementar primero la clase `Interactions`. Esta clase permitirá a la clase `InteractionDetector` llevar la cuenta de las interacciones que se producen entre dos personas. Para hacer esto la clase `Interaction` será capaz de guardar las fechas en las que se producen los encuentros y la duración de los mismos. Para la fecha emplearemos la clase `Date`, que ya está implementada en Java y tiene muchos métodos interesantes (como `isAfter`, `isBefore`, `equals` o `compareTo`). La clase `Interactions` tendrá los siguientes métodos:

d) **[0,5 puntos]** Definir las estructuras de datos adecuadas para implementar la clase `Interactions`

e) **[0,5 puntos]** `addInteraction(LocalDate date, int minutes)` que añade una interacción entre dos personas a la ED. Si la interacción ya existe, se actualiza el número de minutos sumando los nuevos minutos al antiguo.

f) **[1,5 puntos]** `boolean isOfInterest(LocalDate date, int minutes)` que recibe una fecha concreta y los minutos de duración del encuentro y devuelve un valor booleano `true` si alguna de las interacciones registradas es de interés. Una interacción será de interés si la fecha es igual o posterior a la especificada y el número de minutos es mayor o igual al especificado.

NOTA: Se podrán agregar los métodos privados que se consideren oportunos y cualquiera de las estructuras de datos estudiadas durante el curso, valorándose especialmente lo adecuado de la elección que se realice. Sin embargo, se prohíbe modificar de cualquier forma las interfaces de clase suministradas en el proyecto.